

15. Terminal y argumentos

SYS.ARGV + CHAT CONFIGURABLE + PROYECTO INTEGRADOR 1

Programación I
Ingeniería Electrónica y Telecomunicaciones

Comisión 3
Profesor: Lautaro Linquimán
Universidad Nacional de Río Negro



Repaso express

Clase anterior

1. ¿Para qué nos servía crear un ambiente virtual con `venv`?
2. ¿Qué diferencia había entre instalar un paquete en Python global o dentro de `.venv`?
3. ¿Para qué usábamos el archivo `requirements.txt`?

Argumentos de línea de comandos

Hasta ahora estuvimos trabajando con variables definidas dentro del programa o solicitando información al usuario mediante el uso de la función `input()`.

Si bien esto es útil, existen situaciones en las que necesitamos ejecutar nuestros programas con un mayor grado de autonomía. En estos casos, puede resultar poco práctico tener que introducir información manualmente durante la ejecución o modificar el código cada vez que queremos cambiar su comportamiento.

En el mundo del software existe un mecanismo conocido como argumentos de línea de comandos, que nos permite enviar información a un programa en el mismo momento en que lo ejecutamos.

De esta manera, podemos indicarle al programa cómo debe comportarse o con qué datos debe trabajar, sin necesidad de modificar su código ni de ingresar información durante la ejecución.

Primer ejemplo

Desde la terminal podemos ejecutar:

```
python programa.py hola 10
```

El programa recibe:

- `programa.py`
- `hola`
- `10`

sys.argv

`sys.argv` es una lista con los argumentos usados al ejecutar el programa.

```
import sys  
  
print(sys.argv)
```

Ejecutamos:

```
python programa.py hola 10
```

```
['programa.py', 'hola', '10']
```

Estructura de datos

Posición	Contenido	Tipo
0	nombre o ruta del programa	str
1	primer argumento real	str
2	segundo argumento real	str

Aunque escribamos `10`, Python recibe `"10"`.

Todos los argumentos llegan como texto (str).

Problemas variaciones

```
python programa.py  
python programa.py hola  
python programa.py hola 10  
python programa.py hola 10 extra  
python programa.py "Lisa Simpson" 3
```

Preguntas:

- ¿Cuántos elementos tiene `sys.argv`?
- ¿Qué pasa si sobran argumentos?
- ¿Qué creen que pasa cuando usamos comillas?

Rompiendo nuestro programita

```
import sys

nombre_apellido = sys.argv[1]
cantidad = int(sys.argv[2])

for i in range(cantidad):
    print(f"Hola, {nombre_apellido}")
```

¿Qué problemas podría tener este programa?

Validar datos de entrada

Como vimos antes, siempre que un usuario ingrese datos tenemos que validarlos.

1. Validar la cantidad esperada;
2. Mostrar la forma correcta de ejecución;
3. Convertir datos cuando corresponde;
4. Avisar con un mensaje claro si no puede seguir.

Ejercicio 1

Crear `repetir.py`.

El programa recibe un nombre y una cantidad:

```
python repetir.py Maggie 4
```

Debe mostrar:

```
Hola, Maggie  
Hola, Maggie  
Hola, Maggie  
Hola, Maggie
```

Casos para probar: faltan argumentos, sobran argumentos, cantidad inválida.

Archivos como argumentos

Hasta ahora abríamos archivos con rutas escritas dentro del código:

```
archivo = "datos.txt"
```

Pero un programa de terminal suele recibirlas así:

```
python mostrar_archivo.py datos.txt
```

Eso permite usar el mismo programa con distintos archivos.

Manejando archivos desde la CLI

```
import sys

archivo = sys.argv[1]

with open(archivo, "r") as datos:
    contenido = datos.read()

print(contenido)
```

Probamos:

```
python mostrar_archivo.py archivo.txt
python mostrar_archivo.py no_existe.txt
```

¿Observan algun problema con los comandos anteriores?

Validar existencia

Antes de abrir un archivo recibido por argumento, podemos preguntar si existe.

```
import os

archivo = "datos.txt"

print(os.path.exists(archivo))
```

`os.path.exists(...)` devuelve un valor booleano:

- `True`: la ruta existe.
- `False`: la ruta no existe.

Ejercicio 2

Crear `copiar_archivo.py`.

El programa recibe un archivo:

```
python copiar_archivo.py entrada.txt salida.txt
```

Debe:

- validar la cantidad de argumentos;
- comprobar que el archivo de entrada exista si no existe fallar.
- verificar que el archivo de destino NO exista, si existe fallar.
- Leer el archivo de entrada y escribir en el archivo de salida.

Cerrando ejercicio del chat

La IP del servidor como argumento

Cliente configurable

En clase 14 el cliente tenía la dirección escrita en el código:

```
SERVIDOR = "http://localhost:8000"
```

Ahora queremos iniciar el programa así:

```
python cliente.py 192.168.1.50
```

Consigna de cierre:

- recibir la IP por `sys.argv`;
- validar que llegue exactamente un argumento;
- armar la URL del servidor con esa IP;

Proyecto Integrador 1

Parte 1: Conversor TXT → JSON

Repasando consigna